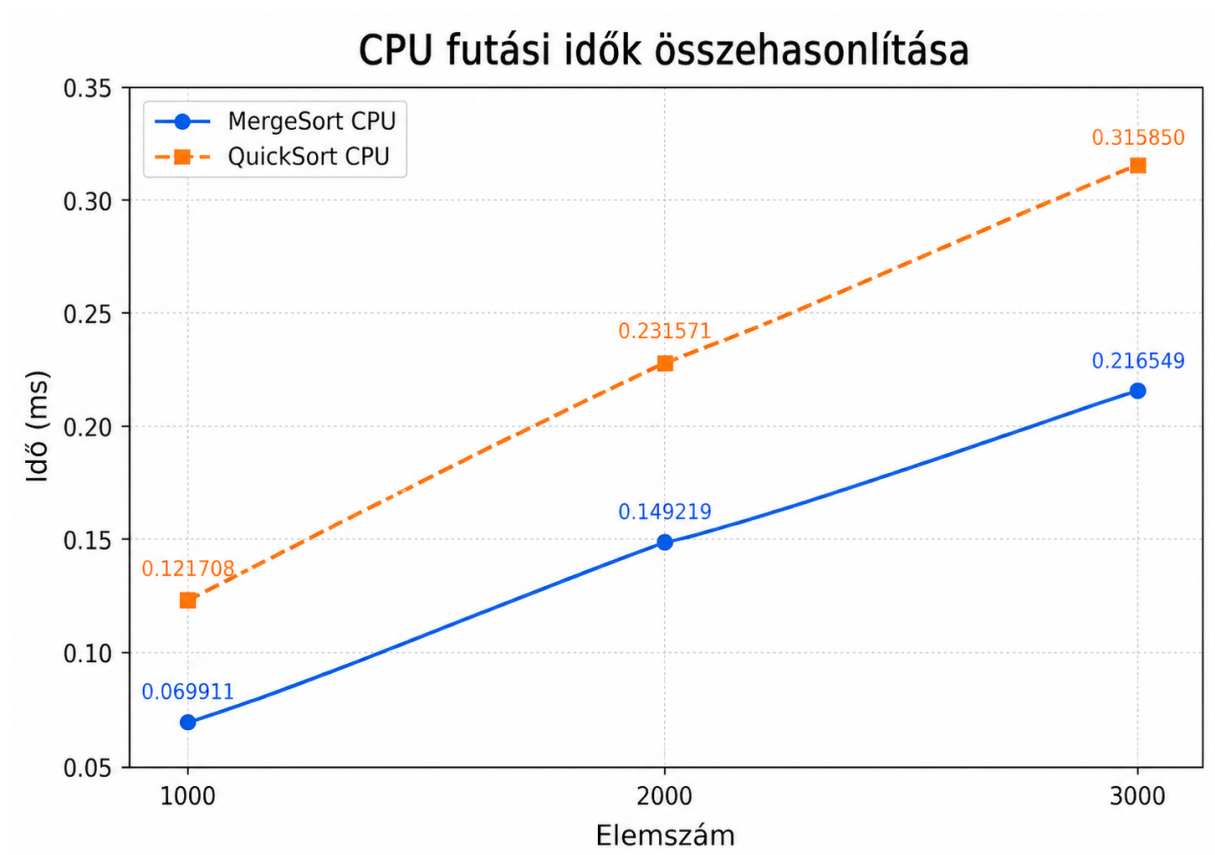


Mérési jegyzőkönyv

1. A mérés célja

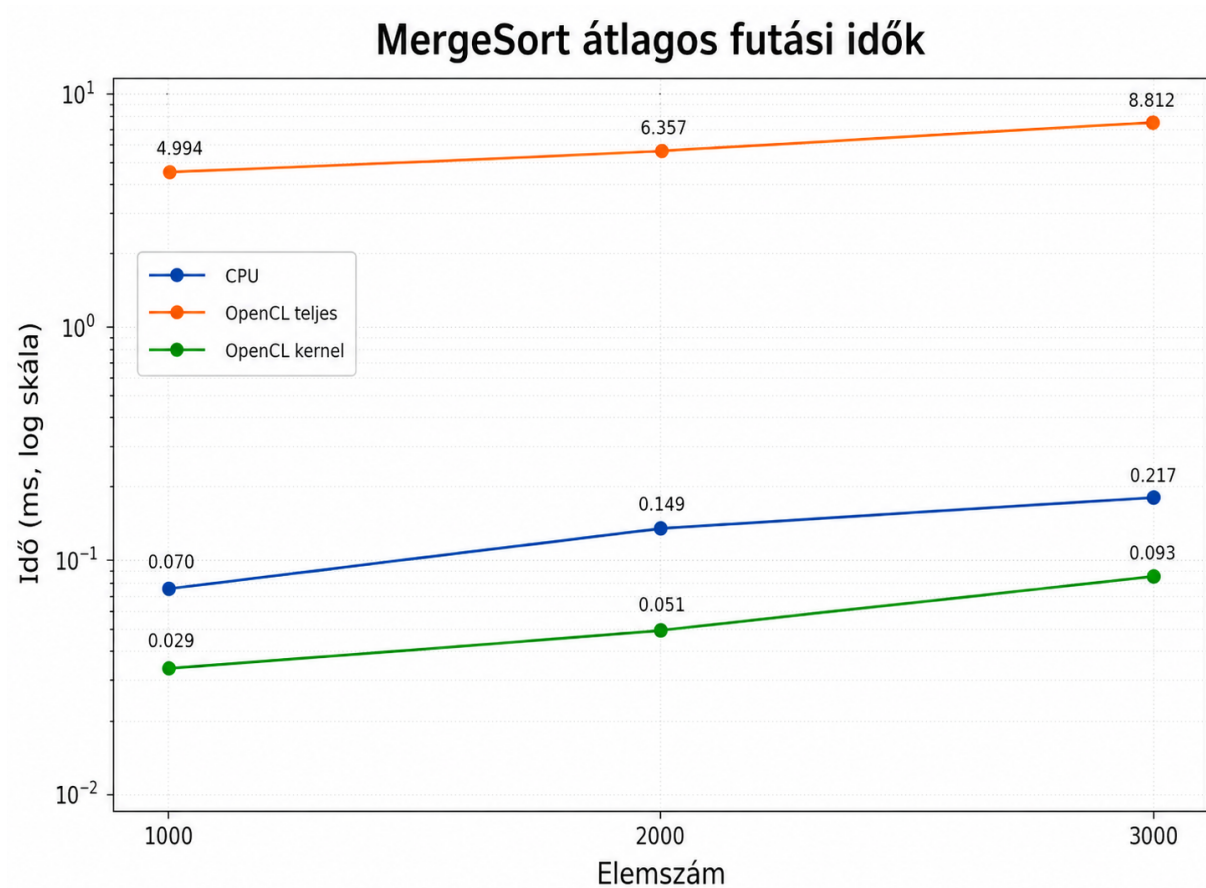
A mérés célja a MergeSort és a QuickSort algoritmusok CPU-s és OpenCL-es megvalósításának összehasonlítása volt. A vizsgálat során azt elemeztem, hogy az egyes algoritmusok hogyan teljesítenek különböző elemszámok mellett, valamint hogy az OpenCL használata jelent-e gyorsulást kis méretű adathalmazok esetén.

KÉPALÁÍRÁSOK



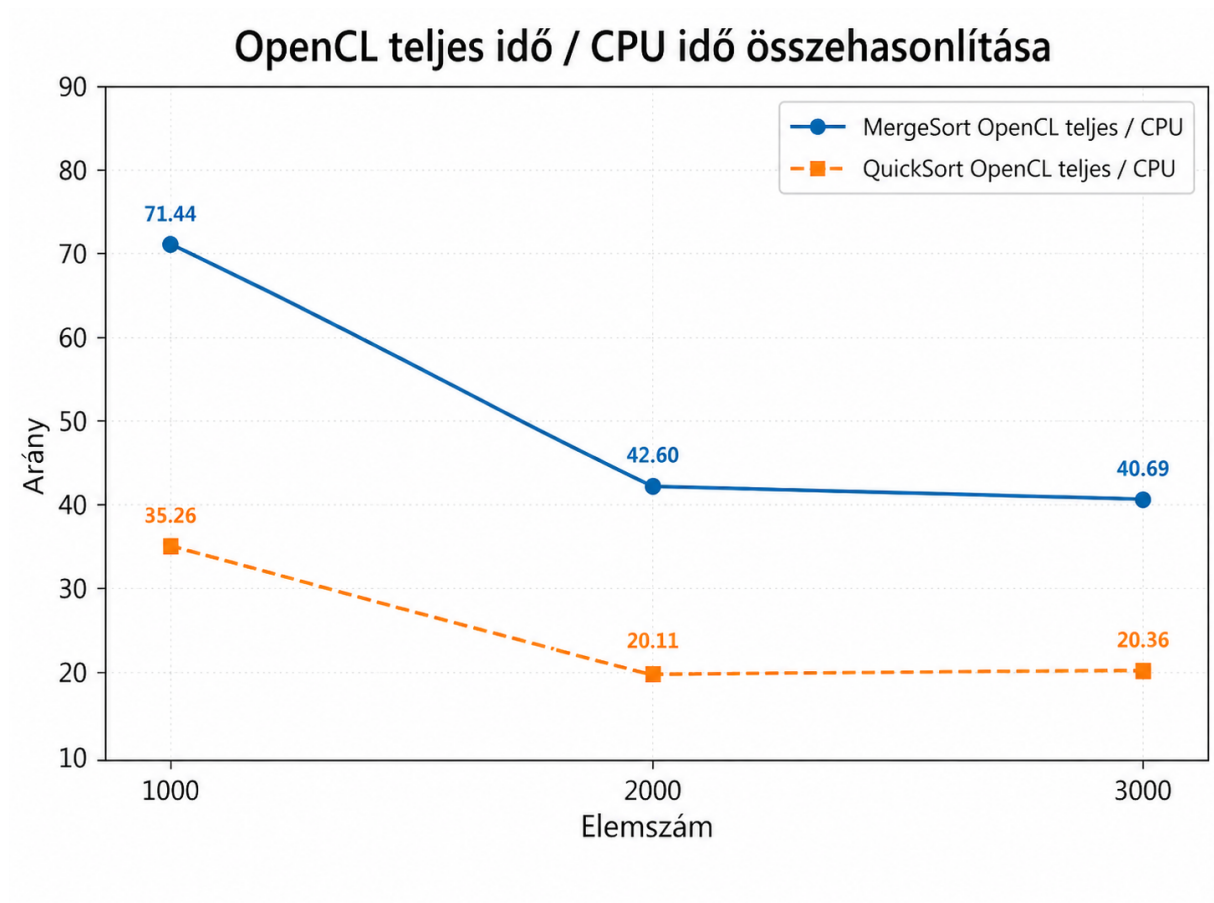
1. ábra: CPU futási idők összehasonlítása

Az ábra a MergeSort és QuickSort CPU-s futási idejét hasonlítja össze 1000, 2000 és 3000 elem esetén. Látható, hogy mindkét algoritmus futási ideje nő az elemszám növekedésével. A mérések alapján CPU-n a MergeSort minden vizsgált elemszámnál gyorsabb volt, mint a QuickSort. Ez abból látszik, hogy a MergeSort értékei végig alacsonyabban helyezkednek el a diagramon.



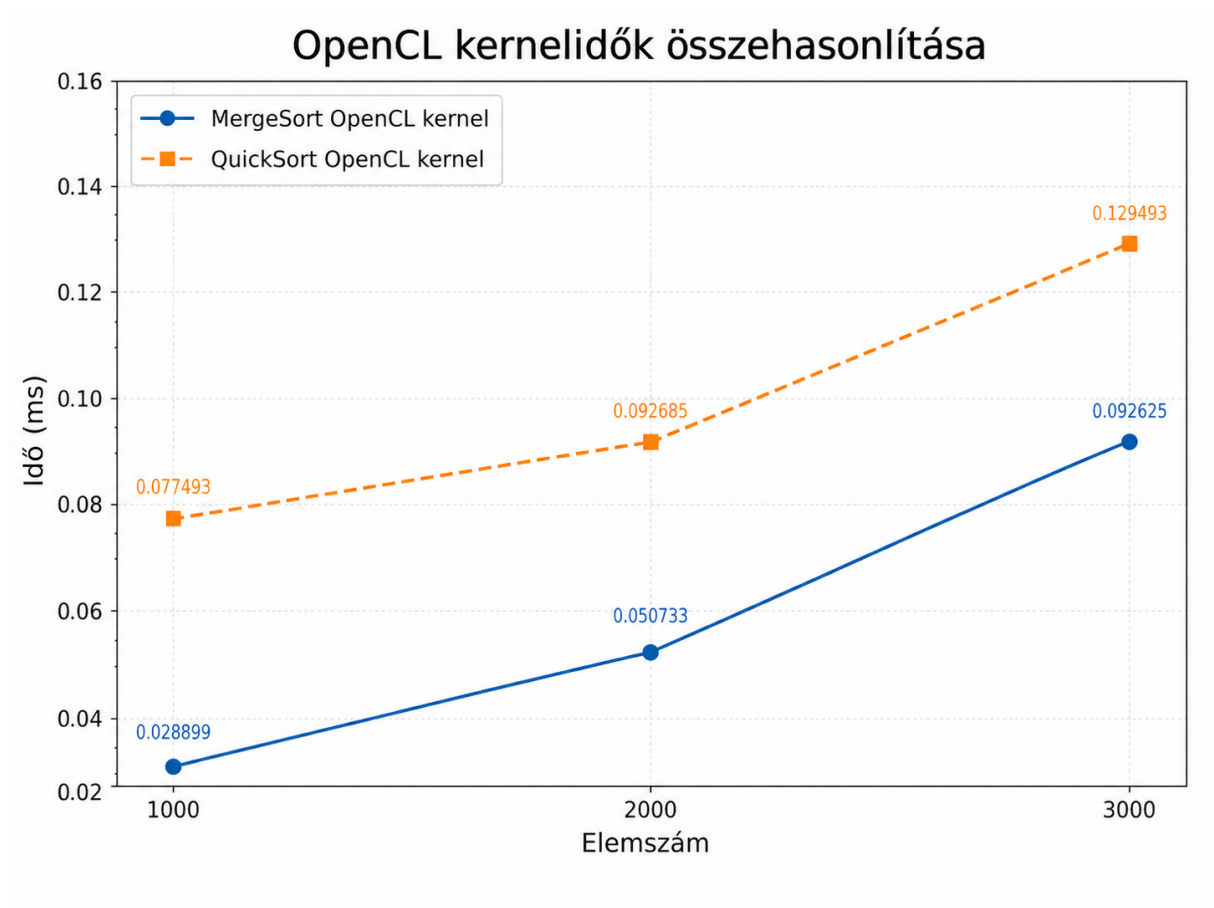
2. ábra: MergeSort átlagos futási idők

Az ábra a MergeSort CPU-s, OpenCL teljes és OpenCL kernel futási idejét mutatja. A diagram logaritmikus skálát használ, mert a teljes OpenCL idő jóval nagyobb, mint a CPU és a kernelidő. Megfigyelhető, hogy maga az OpenCL kernel nagyon gyorsan lefut, viszont a teljes OpenCL futási idő sokkal nagyobb. Ez azt jelenti, hogy a futási idő jelentős részét nem a számítás, hanem az OpenCL környezet használata, az adatmozgatás és az indítási többletköltség adja.



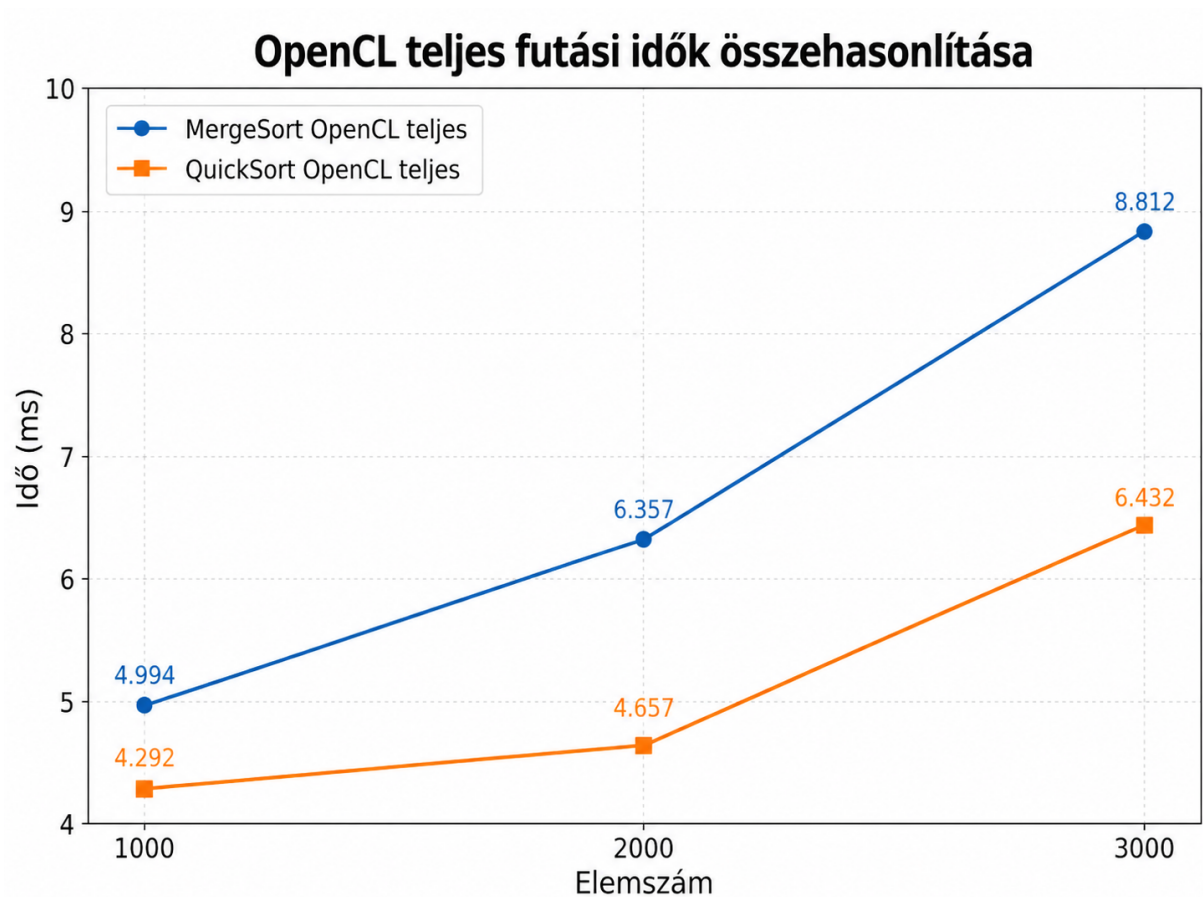
3. ábra: OpenCL teljes idő / CPU idő összehasonlítása

Az ábra azt mutatja meg, hogy az OpenCL teljes futási ideje hányszorosa a CPU-s futási időnek. Mindkét algoritmusnál látható, hogy a vizsgált kis elemszámok mellett az OpenCL teljes futási ideje jelentősen nagyobb, mint a CPU-s futási idő. MergeSort esetén ez az arány 1000 elemnél körülbelül 71-szeres, 3000 elemnél pedig körülbelül 41-szeres. QuickSort esetén az arány kisebb, de még így is 20–35-szörös. Ez alapján kis adatmennyiségnél az OpenCL teljes futása nem előnyös a CPU-hoz képest.



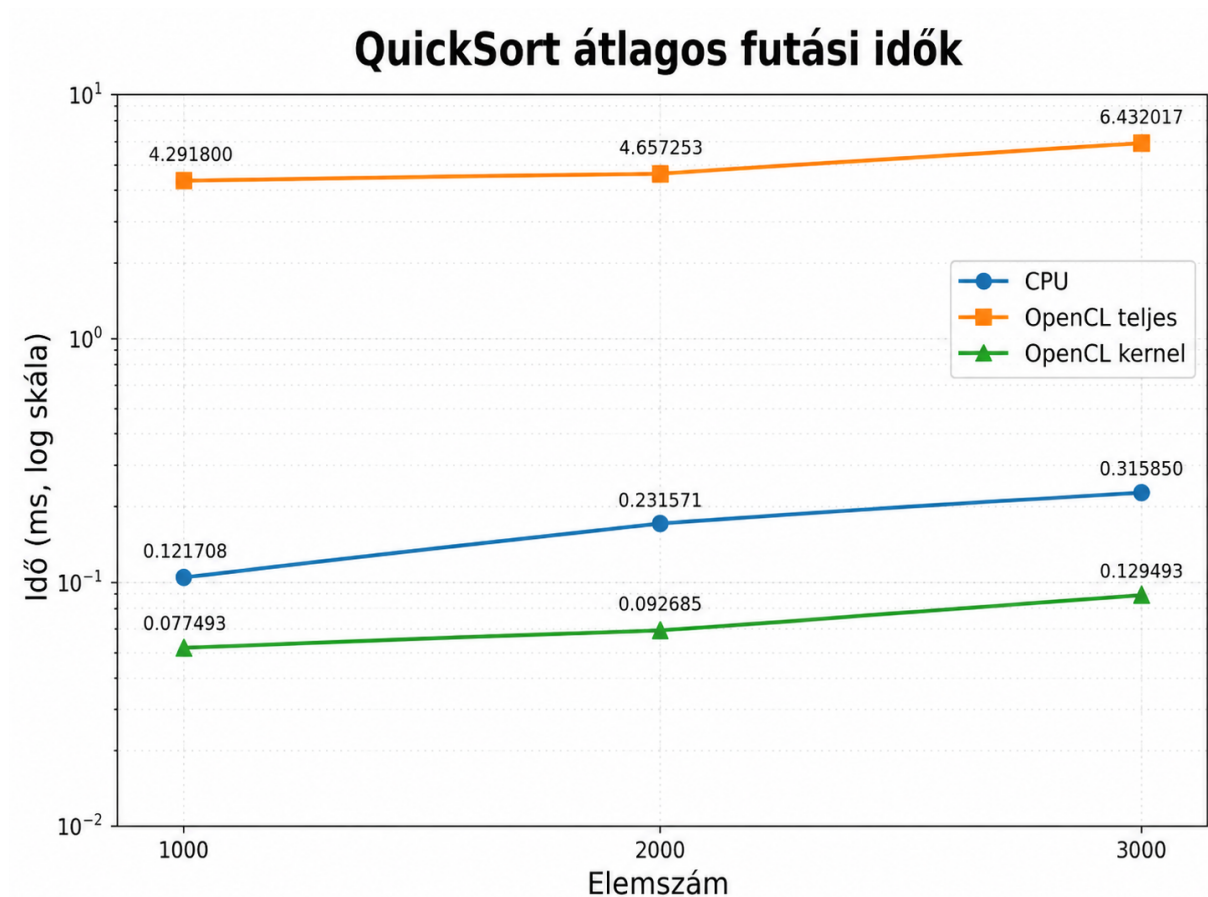
4. ábra: OpenCL kernelidők összehasonlítása

Az ábra a MergeSort és QuickSort OpenCL kernelidejét hasonlítja össze. A kernelidő csak a tényleges GPU-n futó számítás idejét mutatja, nem tartalmazza az adatmásolást és az OpenCL indítási költségeit. A diagram alapján a kernelidők nagyon alacsonyak, ezredmásodperces nagyságrendűek. A MergeSort kernelideje minden elemszámnál kisebb volt, mint a QuickSort kernelideje, viszont mindkét esetben növekedés figyelhető meg az elemszám növelésével.



5. ábra: OpenCL teljes futási idők összehasonlítása

Az ábra a MergeSort és QuickSort teljes OpenCL futási idejét hasonlítja össze. A teljes OpenCL idő tartalmazza a kernel futását, az adatmozgatást, a memóriakezelést és az OpenCL műveletek indítási többletköltségét is. A diagram alapján a QuickSort teljes OpenCL futási ideje alacsonyabb volt, mint a MergeSort teljes OpenCL futási ideje. Ez főleg 3000 elemnél látszik erősen, ahol a MergeSort OpenCL teljes ideje 8,812 ms, míg a QuickSort esetén 6,432 ms volt.



6. ábra: QuickSort átlagos futási idők

Az ábra a QuickSort CPU-s, OpenCL teljes és OpenCL kernel futási idejét mutatja. Itt is logaritmikus skála szerepel, hogy a CPU, az OpenCL teljes idő és a kernelidő egyszerre jól látható legyen. A mérések alapján a QuickSort kernelideje kicsi, viszont a teljes OpenCL futási idő jóval nagyobb, mint a CPU-s futási idő. Ez megerősíti, hogy kis elemszámoknál az OpenCL többletköltsége nagyobb hatással van a teljes futásra, mint maga a párhuzamos számítás előnye.

RÖVID ZÁRÓ KÖVETKEZTETÉS

A mérések alapján megállapítható, hogy a vizsgált 1000, 2000 és 3000 elemszámok mellett a CPU-s megoldások gyorsabbak voltak, mint az OpenCL teljes futási ideje. Ennek oka, hogy ilyen kis adatmennyiségnél az OpenCL használatával járó többletköltségek – például az adatmásolás, memóriakezelés, kernelindítás és szinkronizáció – nagyobbak, mint amennyi időt a párhuzamos végrehajtás meg tudna takarítani.

Fontos különbség látható a teljes OpenCL idő és a kernelidő között. A kernelidők nagyon alacsonyak, tehát maga a GPU-n futó számítás gyors. A teljes OpenCL futási idő viszont jóval nagyobb, ezért a teljes program szempontjából a gyorsulás nem jelentkezik ezeknél a kis elemszámoknál.

CPU-n a MergeSort bizonyult gyorsabbnak, míg az OpenCL teljes futási időket nézve a QuickSort adott kedvezőbb eredményeket. Összességében a mérések azt mutatják, hogy az

OpenCL előnye inkább nagyobb elemszámoknál jelentkezhet, ahol a párhuzamosításból származó nyereség már ellensúlyozhatja az OpenCL indítási és adatmozgatási költségeit.